МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №3

по дисциплине Построение и анализ алгоритмов

Студент гр. 4343 _____ Пимахин Р.А.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм для нахождения расстояния Левенштейна.

Задание

Вариант 9а. Расстоянием Левенштейна называется минимальная стоимость преобразования одной строки в другую с помощью операций вставки одного символа, удаления одного символа и замены одного символа на другой. Необходимо разработать программу поиска минимальной стоимости преобразования строки S в строку T.

В отличие от обычного расстояния Левенштейна, для данного варианта на вход дополнительно подаются индексы нестабильных элементов первой строки. Такие элементы имеют отдельные цены сохранения и замены. Для нестабильных элементов используются специальные стоимости операций: особая цена сохранения и особая цена замены.

Подвариант 9а содержит дополнительное условие: если нестабильным является символ «S», то цена его сохранения равна 0, а особой остается только цена замены

Входные данные:

цены обычных операций замены, вставки и удаления; количество нестабильных элементов; список индексов нестабильных элементов первой строки; особые цены сохранения и замены нестабильных элементов; две строки S и T.

Выходные данные:

Одно число - минимальная стоимость преобразования строки S в строку T.

Описание алгоритма

Для решения задачи используется алгоритм динамического программирования на основе расстояния Левенштейна.

Основная идея алгоритма заключается в построении таблицы dp, где элемент dp[i][j] хранит минимальную стоимость преобразования первых i символов первой строки S в первые j символов второй строки T.

Размер таблицы равен (n+1) * (m+1), где n - длина первой строки, а m - длина второй строки. Начальная инициализация таблицы выполняется следующим образом: dp[i][0] = i * стоимость удаления.

Данное значение показывает стоимость удаления первых i символов первой строки для получения пустой строки.

dp[0][j] = j * стоимость вставки.

Данное значение показывает стоимость получения первых j символов второй строки путем последовательных вставок.

После инициализации таблица заполняется построчно. Для каждой пары символов S[i-1] и T[j-1] рассматриваются три возможных действия:

1. Сохранение или замена символа.

Если символы совпадают, рассматривается операция сохранения. Для обычных элементов ее стоимость равна 0.

Если элемент является нестабильным, используется особая цена сохранения. Для подварианта 9а исключение: если нестабильный символ равен «S», стоимость его сохранения принимается равной 0.

Если символы различаются, выполняется операция замены. Для обычного элемента используется стандартная цена замены, а для нестабильного элемента - специальная цена замены.

2. Удаление символа: dp[i-1][j] + delete_cost

3. Вставка символа: dp[i][j-1] + insert_cost

После вычисления всех вариантов выбирается минимальное значение: dp[i][j] = min(сохранение/замена, удаление, вставка)

После заполнения всей таблицы значение dp[n][m] содержит минимальную стоимость преобразования всей первой строки во вторую.

Оценка сложности алгоритмов

Временная сложность алгоритма: O(n*m) Таблица динамического программирования имеет размер (n+1) × (m+1). Для заполнения каждой ячейки выполняется постоянное количество операций:

1) вычисление стоимости сохранения или замены;

2) вычисление стоимости удаления;

3) вычисление стоимости вставки;

4) поиск минимального значения из трех вариантов.

Так как каждая ячейка обрабатывается один раз, итоговая временная сложность алгоритма составляет O(n*m).

Сложность по памяти: O(n*m)

В реализации используется полная таблица динамического программирования dp размером (n+1) × (m+1), поэтому количество занимаемой памяти зависит от произведения длин входных строк.

Дополнительно в памяти хранятся:

1) две исходные строки S и T - O(n + m);

2) множество индексов нестабильных элементов - O(k), где k количество нестабильных элементов.

Основной вклад в расход памяти вносит таблица dp, поэтому итоговая сложность по памяти равна O(n*m).

Тестирование

Входные данные	Вывод

3 3 4 1 1 4 5 abc adc	7
5 3 5 1 2 4 10 abc adc	6
5 6 7 2 1 2 10 10 qwert zxc	29

Вывод

В ходе лабораторной работы был реализован алгоритм Вагнера—Фишера для вычисления расстояния Левенштейна с учетом нестабильных элементов первой строки. Была добавлена обработка специальных цен сохранения и замены, а также условие для нестабильного символа «S». Тестирование подтвердило корректность работы алгоритма.